

Artificial Intelligence Laboratory

Building and Learning a Bayesian Network with an LLM

Samarth Bhatnagar

October 2, 2026

Contents

1	Introduction	3
2	Bayesian Network Specification	3
3	Reference Bayesian Network	4
4	Exact Inference	5
5	Independent Verification of Inference	5
6	Using an LLM to Generate Bayesian Network Code	6
7	Inspection of LLM-Generated Code	6
8	Problems in the Generated Bayesian Network	7
9	Parameter Estimation from Data	7
10	Maximum Likelihood Estimation	8
11	Effect of Sample Size	8
12	LLM-Generated MLE Code	9
13	Sparse Data and Bayesian Parameter Estimation	9
14	LLM-Generated Bayesian Estimation Code	10
15	Deliberately Invalid Bayesian Network	11
16	Subtle Semantic Error	11
17	Automated Validation	11
18	Additional Inference Queries	12

19 Multiple Datasets	12
20 Failure Modes	13
21 Role of the LLM	13
22 Discussion	14
23 Conclusion	14

1 Introduction

This laboratory investigates the construction, inference, and parameter estimation of a discrete Bayesian network while using a Large Language Model (LLM) as an engineering assistant.

The laboratory distinguishes between two related activities. AI science involves defining a probabilistic model, reasoning about its assumptions, and performing correct inference. AI engineering involves translating that specification into executable software, testing the implementation, and validating the resulting system.

The Bayesian network used throughout the laboratory is

$$C \rightarrow R, \quad C \rightarrow S, \quad R \rightarrow W, \quad S \rightarrow W,$$

where

- C represents Cloudy,
- R represents Rain,
- S represents Sprinkler,
- W represents WetGrass.

The overall engineering workflow is

specification $\rightarrow$ LLM-generated code $\rightarrow$ inspection $\rightarrow$ execution $\rightarrow$ validation $\rightarrow$ testing].

The purpose is not to treat the LLM as a source of probabilistic truth, but to evaluate whether generated software correctly implements a known mathematical specification.

2 Bayesian Network Specification

The Bayesian network factorizes the joint probability distribution as

$$P(C, R, S, W) = P(C)P(R | C)P(S | C)P(W | R, S).$$

All variables are binary, with

$$0 = \text{False}, \quad 1 = \text{True}.$$

The prior probability of Cloudy is

$$P(C = 1) = 0.5.$$

The conditional probabilities for Rain are

$$P(R = 1 | C = 0) = 0.2,$$

$$P(R = 1 | C = 1) = 0.8.$$

The conditional probabilities for Sprinkler are

$$P(S = 1 \mid C = 0) = 0.5,$$

$$P(S = 1 \mid C = 1) = 0.1.$$

The WetGrass probabilities are

$$P(W = 1 \mid R = 0, S = 0) = 0.01,$$

$$P(W = 1 \mid R = 0, S = 1) = 0.90,$$

$$P(W = 1 \mid R = 1, S = 0) = 0.90,$$

$$P(W = 1 \mid R = 1, S = 1) = 0.99.$$

These probabilities form the trusted mathematical specification against which later implementations are compared.

3 Reference Bayesian Network

Before using an LLM, the Bayesian network was implemented directly using the `pgmpy` library.

The graph was constructed with the four directed edges:

```
1 ("Cloudy", "Rain")
2 ("Cloudy", "Sprinkler")
3 ("Rain", "WetGrass")
4 ("Sprinkler", "WetGrass")
```

Listing 1: Bayesian network structure

The CPDs were then specified using `TabularCPD`.

For example, the Rain CPD was defined as

$$P(R \mid C),$$

with columns corresponding to

$$C = 0, \quad C = 1.$$

Similarly, the WetGrass CPD used Rain and Sprinkler as evidence variables. The parent ordering is important because changing the order of the parent states changes the meaning of the columns.

After constructing the model, the following validation was performed:

```
1 reference_model.check_model()
```

Listing 2: Checking the reference Bayesian network

The reference implementation serves as the trusted implementation for the rest of the experiment.

4 Exact Inference

The first inference task was to determine

$$P(R = 1 \mid W = 1).$$

This was performed using the `VariableElimination` algorithm from `pgmpy`.

Variable elimination computes the posterior by combining the relevant factors, summing out hidden variables, and normalizing the resulting distribution.

For this network,

$$P(R, W = 1) = \sum_C \sum_S P(C)P(R \mid C)P(S \mid C)P(W = 1 \mid R, S).$$

The resulting posterior from the reference model was

$$P(R = 1 \mid W = 1) \approx 0.70476923.$$

5 Independent Verification of Inference

To avoid relying on a single software implementation, the posterior was also calculated independently by enumerating all possible assignments of the four binary variables.

There are

$$2^4 = 16$$

possible assignments.

For every assignment, the joint probability was calculated using

$$P(C, R, S, W) = P(C)P(R \mid C)P(S \mid C)P(W \mid R, S).$$

The numerator was obtained by summing the joint probabilities for assignments satisfying

$$R = 1, \quad W = 1,$$

while the denominator was obtained by summing all assignments satisfying

$$W = 1.$$

The resulting value was

$$P(R = 1 \mid W = 1) = 0.70476923.$$

This agrees with the result obtained from `pgmpy`.

Thus, the inference result was independently verified rather than accepted merely because the software executed successfully.

6 Using an LLM to Generate Bayesian Network Code

After establishing the reference implementation, a coding LLM was used to generate Python code for the same Bayesian network.

The model used in the notebook was

Qwen/Qwen2.5-Coder-1.5B-Instruct

The prompt explicitly specified:

- the graph structure;
- the binary state representation;
- all CPT values;
- the requirement to use `pgmpy`;
- the required inference query;
- restrictions on file and network access.

The LLM was asked to return executable Python code.

7 Inspection of LLM-Generated Code

The generated code was not executed immediately.

First, it was extracted from the LLM response and inspected.

An AST-based sanity check was then used to detect certain potentially unsafe operations. The check rejected imports outside the permitted libraries and rejected calls such as

```
eval
exec
compile
open
input
__import__
```

The generated code passed this basic safety check:

Basic check found no listed problems.

However, passing the safety check does not establish mathematical correctness.

This distinction is important because the generated code contained several correctness problems.

8 Problems in the Generated Bayesian Network

The LLM generated code used

```
BayesianModel
```

rather than the current

```
DiscreteBayesianNetwork
```

used by the reference implementation.

More importantly, several CPTs did not correctly encode the specified probabilities.

For example, the Rain CPD generated by the LLM did not preserve the required state interpretation and ordering.

The generated Sprinkler and WetGrass CPDs also differed from the specified CPTs.

Therefore, although the generated program appeared syntactically reasonable, it did not correctly implement the intended probability model.

The notebook therefore kept the execution approval flag as

```
1 APPROVE_GENERATED_CODE = False
```

Listing 3: LLM code approval

and the generated code was not executed.

This was an important experimental result: the LLM produced code that looked plausible and passed the basic AST safety check, but human inspection revealed semantic and API problems before execution.

9 Parameter Estimation from Data

The next part of the laboratory assumed that the graph structure was known but that the numerical CPT parameters had to be estimated from observations.

Synthetic data were generated from the trusted Bayesian network.

A dataset containing

$$N = 1000$$

observations was generated using a fixed random seed.

The resulting dataset had shape

$$(1000, 4)$$

corresponding to the four variables

$$C, R, S, W.$$

The parameters were then estimated using Maximum Likelihood Estimation (MLE). For example,

$$\hat{P}(R = 1 \mid C = 1) = \frac{N(R = 1, C = 1)}{N(C = 1)}.$$

10 Maximum Likelihood Estimation

The graph structure was kept fixed while the CPT parameters were fitted using `DiscreteMLE`.

The fitted model passed

`check_model()`

successfully.

For the Rain parameter,

$$\hat{P}(R = 1 \mid C = 1) = 0.77254098.$$

The manually calculated estimate from the data was also

$$0.77254098.$$

Therefore,

$$\boxed{\text{manual MLE} = \text{pgmpy MLE}}$$

for this parameter.

The true value used to generate the data was

$$P(R = 1 \mid C = 1) = 0.8.$$

The difference between the true value and the estimate is expected because the model was fitted using a finite sample.

11 Effect of Sample Size

The effect of sample size was investigated using

$$N \in \{20, 50, 100, 500, 1000, 5000\}.$$

The estimated values of

$$P(R = 1 \mid C = 1)$$

were:

N	Estimated Probability	Absolute Error
20	0.888889	0.088889
50	0.826087	0.026087
100	0.708333	0.091667
500	0.780488	0.019512
1000	0.772541	0.027459
5000	0.803961	0.003961

Table 1: MLE estimates for different sample sizes.

The true value is

$$P(R = 1 \mid C = 1) = 0.8.$$

The estimates fluctuate for smaller datasets because of sampling variability. With 5000 observations, the estimate was

$$0.803961,$$

which was close to the true value of 0.8.

This illustrates that increasing the number of observations generally reduces the variability of parameter estimates.

12 LLM-Generated MLE Code

The LLM was then asked to generate code that would estimate the CPTs using Maximum Likelihood Estimation while keeping the graph structure fixed.

The generated code again contained problems.

In particular, it used the older

`BayesianModel`

API rather than the current

`DiscreteBayesianNetwork`

model class requested in the prompt.

It also manually specified arbitrary CPDs before performing maximum likelihood estimation, rather than following the requested implementation directly. Thus, although the generated code used the supplied `data` variable, it did not faithfully follow the requested current `pgmpy` workflow.

The generated estimation code therefore was not approved or executed.

The experiment demonstrates that an LLM can produce code that appears conceptually related to the requested task while silently changing important implementation details.

13 Sparse Data and Bayesian Parameter Estimation

The laboratory next considered parameter estimation using a smaller dataset.

A dataset with

$$N = 30$$

observations was generated.

Two estimation methods were compared:

1. Maximum Likelihood Estimation;
2. Bayesian estimation using a BDeu prior.

The Bayesian estimator used

prior type = BDeu

with equivalent sample size

10.

The parameter compared was

$$P(R = 1 \mid C = 1).$$

The results were:

Method	$P(R = 1 \mid C = 1)$
True parameter	0.800000
MLE, $N = 30$	0.909091
BDeu, $N = 30$	0.781250

Table 2: MLE and Bayesian estimates for the small dataset.

The MLE estimate directly reflects the observed frequencies in the small dataset.

The Bayesian estimate combines the observed data with the specified prior. In this experiment, the Bayesian estimate was

0.78125,

while the MLE estimate was

0.909091.

The difference illustrates that Bayesian estimation introduces an explicit prior assumption rather than relying only on observed frequencies.

14 LLM-Generated Bayesian Estimation Code

The LLM was then asked to modify the estimation code to perform Bayesian estimation using a BDeu prior with equivalent sample size 10.

The generated code contained an important error.

Instead of using the existing DataFrame named `data`, the LLM attempted to load a new dataset from

```
your_data.csv
```

This violated the requirements of the task because no external file was supposed to be read.

The generated code also did not correctly follow the required `pgmpy` model-fitting interface.

Consequently, the code was not approved or executed.

This provided another example of why generated code must be inspected against the original specification before execution.

15 Deliberately Invalid Bayesian Network

The laboratory deliberately constructed an invalid CPD to test whether the validation mechanism could detect an inconsistent probability distribution.

For example, one column of the Rain CPD contained probabilities

$$0.9 + 0.3 = 1.2,$$

which is invalid because conditional probabilities must sum to one.

Calling

```
broken_model.check_model()
```

produced:

```
ValueError:
```

```
Sum or integral of conditional probabilities for node Rain  
is not equal to 1.
```

Thus, the library correctly detected the invalid model.

16 Subtle Semantic Error

A more subtle error was then introduced by permuting the WetGrass probabilities while maintaining valid probability sums.

In this case, every column of the CPD remained normalized, meaning that the model could pass

```
check_model()
```

even though the probabilities corresponded to incorrect parent-state configurations.

This demonstrates an important distinction:

valid probability values $\neq$ correct probability semantics

A model can therefore be mathematically normalized and syntactically valid while still representing the wrong probability distribution.

For this reason, semantic tests and independently known inference results are required in addition to structural validation.

17 Automated Validation

The notebook converted important correctness requirements into automated tests.

The reference model was checked for:

- the expected variables;
- the expected directed edges;
- valid CPDs;

- agreement between exact inference and independent enumeration.

The reference test suite completed successfully:

All reference tests passed.

This illustrates how mathematical expectations can be converted into repeatable software tests.

18 Additional Inference Queries

The notebook also evaluated several additional inference queries using the trusted Bayesian network:

$$P(S = 1 \mid W = 1),$$

$$P(C = 1 \mid W = 1),$$

and

$$P(R = 1 \mid W = 1, S = 0).$$

These queries were used to further exercise the inference implementation with different query variables and evidence configurations.

The resulting probabilities were:

Query	Probability
$P(S = 1 \mid W = 1)$	0.42784615
$P(C = 1 \mid W = 1)$	0.57461538
$P(R = 1 \mid W = 1, S = 0)$	0.99220220

Table 3: Additional inference queries evaluated using the trusted model.

These results demonstrate that the same trusted Bayesian network can answer different posterior queries by changing the query variables and the available evidence.

19 Multiple Datasets

The laboratory also considered the effect of random sampling by generating several datasets using different random seeds.

For each dataset, an MLE model was fitted and

$$\hat{P}(R = 1 \mid C = 1)$$

was recorded.

Different datasets generated from the same underlying Bayesian network can produce different parameter estimates because the observations are random samples.

This variation represents sampling variability and does not imply that the underlying Bayesian network or the estimation library is inconsistent.

20 Failure Modes

Several possible failure modes were identified during the laboratory.

- The natural-language specification may be ambiguous.
- An LLM may use an obsolete library API.
- Parent or state ordering in a CPD may be reversed.
- The generated code may contain incorrect probability values.
- The query variable or evidence may be specified incorrectly.
- The generated program may modify experimental data instead of using the supplied dataset.
- A syntactically valid program may still represent the wrong probability distribution.
- Sparse datasets can lead to unstable parameter estimates.
- A Bayesian prior may be introduced without being explicitly documented.

These failure modes show why LLM-generated scientific code requires both software validation and domain-level reasoning.

21 Role of the LLM

The laboratory separated the responsibilities of the different components.

Task	Responsible component
Interpret natural-language programming request	LLM
Generate candidate Python code	LLM
Represent Bayesian network	pgmpy
Check local model consistency	pgmpy
Perform exact inference	Variable Elimination
Estimate CPTs	MLE / Bayesian estimators
Validate generated implementation	Tests and independent checks
Determine whether the model is scientifically correct	Human reasoning

Table 4: Separation of responsibilities in the laboratory.

The LLM was useful for producing candidate implementations, but the experiments demonstrated that its output could contain obsolete APIs, incorrect CPTs, and violations of the original specification.

Therefore, the LLM was treated as an engineering assistant rather than as an authority on the probabilistic model.

22 Discussion

The most important observation from the laboratory was that successful program execution is not sufficient evidence of scientific correctness.

The generated inference code passed the basic AST safety check, but inspection revealed that it did not correctly represent the specified Bayesian network. Similarly, the generated MLE and Bayesian-estimation programs contained deviations from the experimental requirements.

The reference implementation provided an independent standard against which these generated programs could be evaluated.

The experiment with the deliberately broken CPD showed that structural validation can detect some classes of errors. However, the semantic error experiment demonstrated that a model can pass normalization checks while still representing the wrong probability distribution.

Consequently, reliable AI engineering requires several levels of validation:

$$\boxed{\text{syntax} \rightarrow \text{safety} \rightarrow \text{structure} \rightarrow \text{parameters} \rightarrow \text{semantics} \rightarrow \text{scientific result}}$$

23 Conclusion

This laboratory provided hands-on experience with Bayesian networks, exact inference, maximum likelihood estimation, Bayesian parameter estimation, and the use of an LLM as a coding assistant.

A trusted Bayesian network was first constructed manually and used to obtain the reference posterior

$$P(R = 1 \mid W = 1) \approx 0.70476923.$$

The result was independently verified through exhaustive enumeration of the possible binary assignments.

Synthetic data were then generated from the reference model and used for maximum likelihood estimation. For a dataset of 1000 observations, the estimated value of

$$P(R = 1 \mid C = 1)$$

was

$$0.77254098,$$

compared with the true value of 0.8. The estimate was also verified manually from the observed counts.

Experiments with different sample sizes demonstrated sampling variability and showed that estimates generally become more stable as the amount of data increases.

For a small dataset of 30 observations, MLE and BDeu Bayesian estimation produced different estimates. This illustrated the effect of incorporating a prior into parameter estimation.

The LLM experiments demonstrated both the usefulness and limitations of LLM-assisted programming. The model was able to generate candidate Bayesian-network and

parameter-estimation code, but several generated programs contained outdated APIs, incorrect CPT semantics, or violated the stated experimental requirements.

The central lesson of the laboratory is therefore:

LLM-generated code must be inspected, tested, and independently validated.

The complete engineering workflow is

specify $\rightarrow$ generate $\rightarrow$ inspect $\rightarrow$ execute $\rightarrow$ validate $\rightarrow$ test.

An LLM can accelerate implementation, but responsibility for the correctness of the probabilistic model, the assumptions, and the final scientific conclusions remains with the engineer.